# CS114 Final Project Report
# Particle System Based Fire Simulation

Sharon Leo

## 1. Introduction

Realistic fire simulation is applicable in many scenarios, in both computer graphics and in real world simulation. Fire is complex in nature, and due to its natural chaos can be difficult to simulate, thus many simplifications are enacted to replicate a similar motion and render a semi-realistic motion of fire. In this project, I intend to create a simplified fire simulation via a particle system with implementation details from Liu, Xumin et al.[2]. The language used is Python with the NumPy library to maintain the numerical values, OpenGL library to render the simulation, and PyQT.6 library to display the render.

## 2. Particle System

Although real fire has an irregular shape, in order to simply simulate the flame, a particle system is used. The fire consists of a series of particles that update its state per specified time frame. To be able to update its location Each particle must maintain its position, velocity, and acceleration in 3D space. Several forces are applied upon each particle, including gravity, buoyancy, and external forces. Along with its position, other aspects such as lifespan, size, color, and transparency are also maintained and updated. These values for each particle are maintained through a class object.

### 2.1. Particle Initialization

Each particle maintains its position, velocity, and acceleration in 3D space. This is maintained using numpy arrays. In order to simulate the chaoticness of a flame, the initial position is set to a random location within the radius of the flame. This location is calculated by choosing a random position in the spherical coordinate system[3] by generating $\xi_1$, $\xi_2$, and $\xi_3$ drawn independently from a uniform distribution between [0, 1) and calculating $\theta$, $\phi$, and distance such that:

$$\Theta = 2\pi\xi_1$$

$$\phi = arccos(1 - 2\xi_2)$$

$$distance = radius\ \sqrt{\xi_3}$$

Once calculated, these values are converted into the polar coordinates x, y, z such that:

$$x = x_{center} + distance * sin(\phi) * cos(\Theta)$$

$$y = y_{center} + distance * sin(\phi) * sin(\Theta)$$

$$z = z_{center} + distance * sin(\phi)$$

Additionally, to provide the buoyancy of the flames, each particle is given an initial upward velocity between 0.5 and 1.5. As there are no external forces acting upon the particle at the start, acceleration is set to 0.

Each particle is given an initial color of yellow. The particle also maintains an initial and current size varying from 0.02 to 0.05 radius.

As the particles do not remain in simulation indefinitely, a lifespan is tracked to determine when the particle has died out, with its initial value varying between 1.5 and 2 seconds. Simultaneously, the current age of the particle is maintained, initially at 0.

### 2.2. Particle Update

Every clock tick, the particle's state is updated. The acceleration is determined by the net forces applied on the particle at a given time. A slight downward gravitational force of 0.5 is applied to the y acceleration on each particle. Each particle of the flame does not have a straight upward movement. To replicate this, some randomized turbulence is added to each flame in the x- and z-axes by adding some slight acceleration between -0.5 and 0.5 in those directions.

The velocity is then updated such that:

$$v = v_0 + a * time$$

and the position of the particle is updated such that:

$$p = p_0 + v * time$$

The acceleration, velocity, and position of the particles are represented by a 3 dimensional vectors $(x, y, z)$.

To simulate a spark burning out, the remaining lifespan of each particle is decreased by the timestep while the age is increased by the timestep.

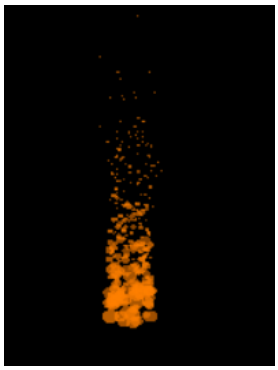

Figure 1: A frame from the initial resulting fire simulation

Initially, the color of the particle was set to constant orange color, as seen in Figure 1, however a flame typically starts as a bright yellow color while the flame's tip is more orange. To replicate this, the flame's color is calculated based on the remaining lifetime such that current color of the particle is:

$$color = yellow * remainingLife + orange * (1 - remainingLife)$$

To replicate the flickering effect of the sparks, the transparency of the particle is continuously reassigned a value between 0.5 and 0.8 and is divided by an alpha level that controls the amount of glowing where 1 is the most opaque. This ensures that the particles become more opaque when there is less glow.

The particles gradually decrease in size throughout the simulation, so the size of the particle is proportional to its remaining lifespan such that the size of the particle reaches 0 when the remaining lifespan is 0. Once the particle has reached a size of 0 or has no more remaining lifespan, the particle is listed as being dead.

## 3. Fire System

All the particles of the flame need to be tracked to create a cohesive object. Additionally, they needed to be updated based on the current time of the simulation and the forces being applied. The state of the fire is maintained with a class object.

### 3.1. Particles in Simulation

An array is used to maintain each particle in the simulation, with a maximum number of particles initialized to 100 and radius initialized to 0.15. Additionally, the center point of the flame is set to (0, 0, 0). To give the particles a glowing effect, each particle is duplicated with a doubled size. This replicates the glowing by having the center of the particle more opaque than the edges. This alpha level is initially 3, creating two layers of transparent glowing.

The fire object is initialized and particles are created one by one until it reaches the max number of particles. For each frame of the simulation, each existing particle's state is updated. If a particle is found to be dead, it is removed from the system and immediately replaced. This animation loop is updated and displayed via PyQT's GLWidget class. Initially, the display showed blank, so a slight translation was used to put the camera behind the flame, with the final rendering as seen in Figure 2.

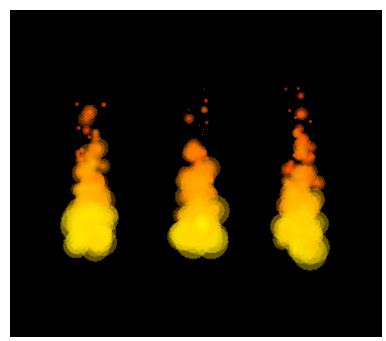

Figure 2: Several frames from the final rendering of the fire simulation

### 3.2. Wind

To show some more movement of the flame, a simple external force of wind was applied to the system. This would add a positive force value of 5 to the acceleration of the particle in the x-direction.

In order to make the wind more interesting, the y-position of the wind was calculated based on a sinusoidal wave with time as the x-axis where:

$$y = amp * sin(freq * time)$$

The wind was given a size of 0.2 such that it affected all particles 0.1 above and below its current y-position. To visualize the current location of the wind, a sphere is drawn to the side of the flame following the same y-position. This provided the results seen in Figure 3.

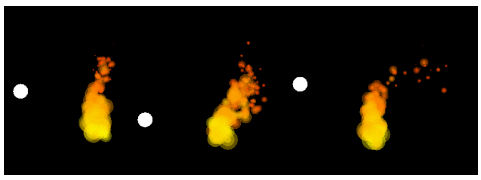

Figure 3: Several frame including the wind force acting on the flame

**4. User Interface**

A user interface was constructed with the PyQT library to allow the user to interact with different values. This included toggling rotation and wind and adjusting values for fire radius, number of particles, and amount of glow. When wind is enabled, it also allows adjustments to the wind's speed. The original codebase for the window widgets were referenced from Anchor the Educator's video tutorial[1]. Toggles were created with checkboxes and values were created with sliders, resulting in the display seen in Figure 4.

The minimum and maximum amounts of values needed to be considered due to heavy amounts of rendering slowing down the computations. The following are the limits of the values for the interface:

| | minimum | maximum |
|---|---|---|
| Wind speed | 1 | 10 |
| Fire radius | 0.05 | 5 |
| Number of particles | 50 | 400 |
| Alpha level | 1 | 5 |

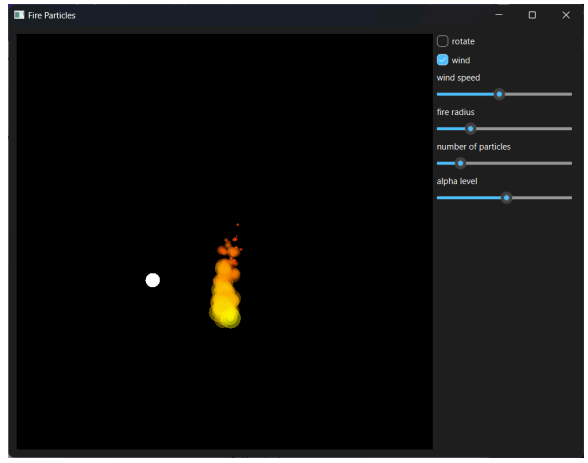


Figure 4: User interface with fire simulation

**5. Conclusion**

Replicating the behavior of a flame has some difficulties due to its natural chaoticness. A particle system was chosen to assist with the ease, but additional techniques could be applied that replicates a more silky connection between sparks.

Additionally, as more particles are rendered, both by number and glow, as well as when wind is added, the rendering speed decreases. Some further considerations on balancing efficiency and flexibility can be made, including considering alternate languages that are faster in computation.

**6. References**

Language: Python

Libraries used:

1. NumPy
2. PyOpenGL
3. PyQt.6

Sources:

1. Anchor the Educator. (2014, Mar 22). Let's Learn Python #25 - UI with PyQt & OpenGL [Video]. YouTube. https://www.youtube.com/watch?v=eJveTXSXs8Q
2. Fitzpatrick, M. (2025, March 19). PyQt6 tutorial 2024, create python guis with Qt. Python GUIs. https://www.pythonguis.com/pyqt6-tutorial/

3. International Journal of Control and Automation Vol. 7, No. 12 (2014), pp. 23-36 http://dx.doi.org/10.14257/ijca.2014.7.12.03
4. M. Balci and H. Foroosh, "Real-time 3D fire simulation using a spring-mass model," 2006 12th International Multi-Media Modelling Conference, Beijing, China, 2006, pp. 8 http://dx.doi.org/10.1109/MMMC.2006.1651309
5. PyOpenGL documentation. (n.d.). https://pyopengl.sourceforge.net/documentation/index.html
6. Weisstein, Eric W. "Sphere Point Picking." From MathWorld--A Wolfram Web Resource. https://mathworld.wolfram.com/SpherePointPicking.html